SOFTWARE REQUIREMENTS SPECIFICATION

for

CAMPUSGIG

A University-Restricted Gig Marketplace

Version 1.0

Prepared by Group 2:
Shashwat Narwal (1024240045)
Mohd Arham (1024240051)
Krish Gakhar (1024240060)
Ineshvir Singh (1024240067)

BE Third Year, CSE

Submitted to : Anushka
Computer Science and Engineering Department
Thapar Institute of Engineering and Technology

October 13, 2026

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Intended Audience and Reading Suggestions . . . 3
- 1.3 Project Scope . . . 5

2 Overall Description **8**
- 2.1 Product Perspective . . . 8
- 2.2 User Classes and Characteristics . . . 8
- 2.3 Product Functions . . . 10
- 2.4 Operating Environment . . . 14
- 2.5 Design . . . 15

3 System Features **21**
- 3.1 Description and Priority . . . 21
- 3.2 Functional Requirements . . . 22

4 Other Nonfunctional Requirements **34**
- 4.1 Performance Requirements . . . 34
- 4.2 Security Requirements . . . 35
- 4.3 Software Quality Attributes . . . 36
- 4.4 Business Rules . . . 37

5 Other Requirements **39**
- 5.1 Constraints . . . 39
- 5.2 Assumptions and Dependencies . . . 39
- 5.3 Deferred Requirements . . . 40
- 5.4 Maintenance . . . 41

1 Introduction

1.1 Purpose

This document specifies the software requirements for **CampusGig**, a gig marketplace restricted to students of Thapar Institute of Engineering and Technology. It states what the system must do, who uses it, what data it holds, how it is secured, and how each requirement will be verified.

Students on a campus already exchange small pieces of paid work: a poster before a society deadline, two hours of tutoring before a quiz, a photographer for one evening. Today this happens over group chats, which reaches only those already in the right group and leaves no record of what was agreed. CampusGig replaces that with a web application where a student posts work with a budget and a deadline, other students search the open postings and apply, and the poster decides who gets it. Membership is the defining constraint: only an account registered with an `@thapar.edu` address can use the system, so both sides of a gig know the other is a real student of the institute.

This is a requirements specification, not a progress report. It describes the complete platform and states the implementation status of every requirement explicitly.

1.2 Intended Audience and Reading Suggestions

- **Course faculty and evaluators**, to assess scope, feasibility and verifiability. Chapters 1 and 2 give the complete picture; the traceability table at the end of Section 3.2 shows how each requirement is checked.
- **The development team (Group 2)**, to plan the remaining milestones. Chapters 3 and 5 are the working chapters.
- **Testers**, to derive test cases from requirement identifiers, using that same traceability table.
- **Future maintainers**, to understand the data model, the authorization rules and the extension points.

Readers new to the project should read Section 1.3 for the scope, Section 2.2 for the user classes and permission matrix, and Section 2.5 for the workflows and the API contract. Every functional requirement has an identifier of the form FR-$f.n$, where f is the feature number from Table 3.1; non-functional requirements use NFR-n and business rules BR-n. Each carries a priority and an implementation status, both defined in Section 3.1.

Definitions and abbreviations

Table 1.1: Definitions and abbreviations used in this document

Term	Meaning
SRS	Software Requirements Specification
Gig	A unit of paid work offered by one student, with a title, description, category, budget and deadline
Publisher	The student who created a given gig. A capacity, not an account type
Worker	The student doing the work on a given gig. A capacity, not an account type
Application	A student's request to be given a gig, submitted at the posted budget. Not a priced bid
Award	The poster's act of accepting one applicant, which closes the gig and declines the rest
Session	A server-side record of a logged-in browser, referenced by a token held in a cookie
HttpOnly cookie	A cookie that scripts running in the page cannot read, which is what protects the session token from theft by injected script
Institutional email	An address whose domain is exactly `thapar.edu`
Contract	A planned record created when a gig is awarded, carrying the agreed amount and a lifecycle state
Escrow	Holding the agreed amount so it is committed but not yet paid out
Hold	The state of funds moved from a poster's available balance to their held balance
Ledger	An append-only double-entry record of value movements
Balanced transaction	A ledger transaction whose entries sum to zero
Idempotency key	A client-supplied key that lets a retried request return the original result instead of performing the operation again
Deliverable	A file submitted as the output of a contract, recorded with a content hash
Dispute	A raised disagreement over a contract, during which funds stay held
Reputation summary	A derived figure recomputed from settled contracts, never an independent source of truth
Milestone	One of the eight planned delivery stages of the platform
Paise	The smallest unit of Indian currency. All planned monetary amounts are whole numbers of paise
CRUD	Create, read, update, delete
DFD	Data flow diagram

References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications.*

2. Group 2, *CampusGig Project Proposal*, Thapar Institute of Engineering and Technology, September 2026.

3. Group 2, *CampusGig Architecture Document* (`ARCHITECTURE.md`), project repository.

4. Group 2, *CampusGig Project Plan* (`PROJECT_PLAN.md`), project repository.

5. Group 2, *CampusGig UML Model Set*, prepared for the Software Engineering laboratory, September 2026.

6. FastAPI, SQLAlchemy, Alembic, Pydantic, PostgreSQL and React documentation.

7. Open Worldwide Application Security Project, *Session Management Cheat Sheet* and *File Upload Cheat Sheet*.

1.3 Project Scope

CampusGig gives students a single place to post small jobs, find them, apply to them and manage the outcome. The platform approved in the project proposal has eight milestones: the first three are built, the remaining five are specified here.

Objectives

1. Restrict all access to verified institutional accounts, enforced on the server rather than in the browser.

2. Provide authenticated sessions that can genuinely be revoked, so that logging out ends the session on the server.

3. Provide a searchable gig board with filtering, sorting and pagination, and let any student post to it.

4. Implement the application workflow end to end, so that a gig can be applied to, awarded to exactly one student, and marked complete.

5. Give every student a profile carrying the information a poster needs to judge an applicant.

6. Protect payment through escrow and a double-entry ledger, so that neither side of a gig has to pay first on trust.

7. Make every disputed outcome resolvable from recorded evidence rather than from recollection.

8. Verify every rejection path with an automated test that runs against the real database schema.

In scope (current build)

- Registration restricted to the `@thapar.edu` domain, with password rules, duplicate detection and login lockout.
- Login, logout and server-side sessions carried in an HttpOnly cookie.
- Posting a gig; browsing open gigs with free text search, category, budget range and deadline filters, four sort orders and pagination.
- Viewing a gig, applying with a message, and as the poster accepting, declining or completing an applicant.
- A profile with a picture, a resume in PDF form, a short description and read-only signup details.
- A consistent error contract, health endpoints and a generated API document.

In scope (specified, to be implemented)

- Contracts with an enforced state machine and an append-only status history.
- Escrow backed by a double-entry ledger, with idempotency keys and row locking on every money-moving operation.
- Deliverable upload to object storage with a recorded content hash.
- Dispute raising and administrative resolution from an evidence timeline.
- Ratings and a reputation summary recomputable from settled contracts.
- Administrator tooling for takedowns and ledger reconciliation.

Out of scope

- Any real payment gateway, bank integration or movement of actual money. The ledger models value; it does not settle it externally.
- Integration with any institute system. The email domain is an identity signal only; the system neither sends nor receives email.
- Email, SMS or push notification delivery. Any later notification is in-app.
- Mobile applications; the system is a responsive web application only.
- Multi-institution support; the allowed domain is a single configured value.

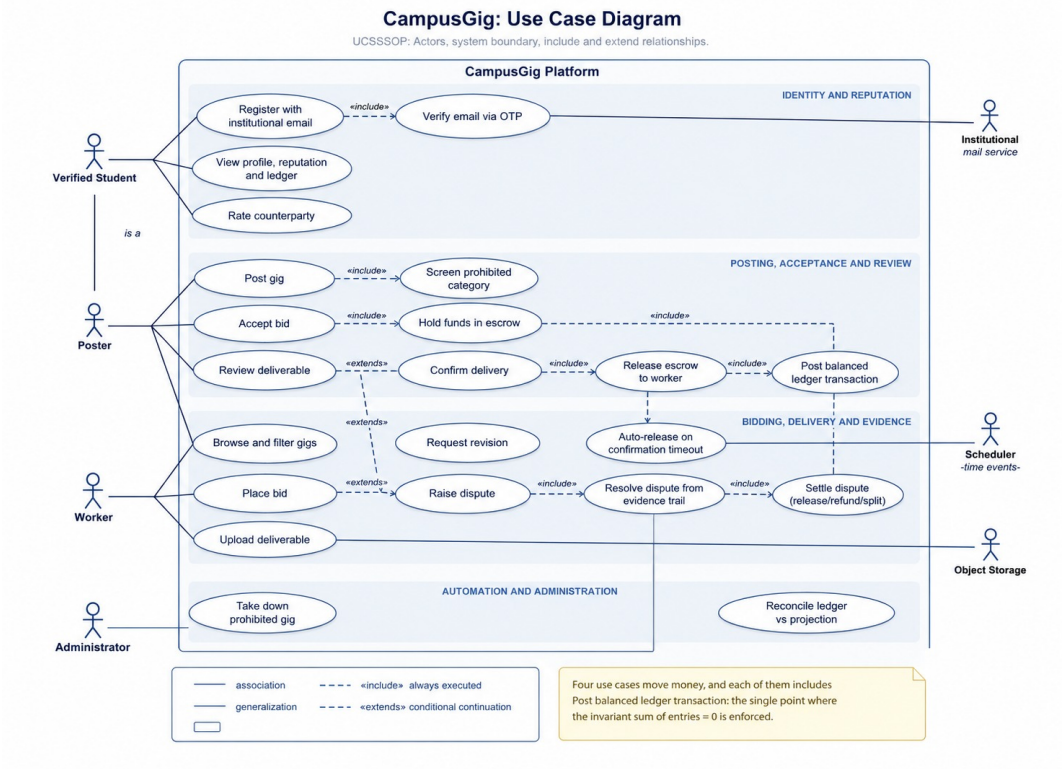


Figure 1.1: Use case diagram for the CampusGig platform

Figure 1.1 shows the scope of the whole platform. Three external actors sit outside the boundary: the institutional mail domain that owns student identity, an object store for deliverable files, and a scheduler firing time-based events such as the expiry of a confirmation window. The human actors are a Verified Student, generalising into a Poster and a Worker, and an Administrator. Poster and Worker are capacities rather than account types, since the same person is a poster on a gig they created and a worker on one they applied to; this is why nobody chooses a role at signup and the data model has one student account type.

An `include` marks a step that always runs within its parent use case, such as screening the category whenever a gig is posted; an `extend` marks a conditional continuation, such as requesting a revision after a review. Four use cases move money, and each includes *Post balanced ledger transaction*, the single place where the accounting rule is enforced.

2 Overall Description

2.1 Product Perspective

CampusGig is a new, self-contained product. It replaces an informal process carried out over chat applications rather than an existing system, so it has no legacy data to migrate and nothing it must stay compatible with.

The system is a **modular monolith**: two deployable pieces and a database.

1. **Presentation layer:** one React single-page application holding the signup and login pages, the gigs board, the gig detail page, the applied page and the profile page.
2. **Application layer:** a FastAPI service in four strictly one-directional layers. Routers handle HTTP only; services hold the business rules and import nothing from the web framework; repositories contain every database query in the project; models are the only place a table is declared. A router never builds a query, and a service never raises an HTTP exception, raising instead a domain error that a single handler maps to a status code.
3. **Data layer:** PostgreSQL through SQLAlchemy with Alembic migrations. The browser never reaches the database. Uploaded files are written under the `MEDIA_ROOT` setting and served back under `/media`.

Two properties follow from this layering: every error has the same shape because exactly one place builds it, and all filesystem access sits in one module, so moving to object storage changes that file alone.

Figure 2.1 places CampusGig among the parties it exchanges data with. A Verified Student supplies registration details, credentials, gigs, applications and profile files, and receives a session cookie, gig listings, applicant lists and errors. Dashed flows are not yet implemented: the Administrator has no feature behind it, and the mail domain is shown as the source of the identity guarantee only, since the system performs no live lookup against it and sends it no mail. The object store and scheduler serve the planned deliverable and timeout features.

2.2 User Classes and Characteristics

CampusGig has three user classes.

- **Student**, any holder of a valid `@thapar.edu` address. This is the majority of users and the only class created by self-service registration; students are assumed

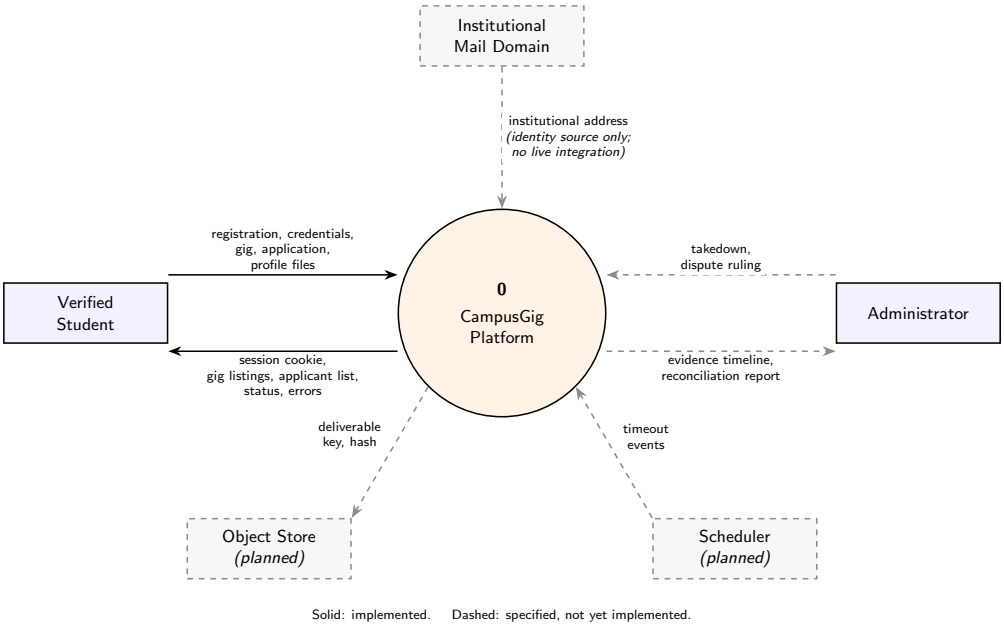


Figure 2.1: Context diagram (DFD level 0)

comfortable with ordinary web applications and need no training. One account carries both capacities at once:

  - *Poster*, when acting on a gig they created.
  - *Worker*, when acting on a gig they applied to.

  The system decides which rules apply by comparing the requester against the owner of the gig in question, never by reading a stored role.

- **Administrator**, a privileged account for takedowns, dispute resolution and ledger reconciliation. The role exists in the data model, but no administrator-only feature is implemented, and the role can never be obtained through registration because the request has no field for it.
- **Unauthenticated visitor**, who can reach the signup and login pages and nothing else. Every other page and data endpoint rejects a request without a valid session.

Table 2.1 states the authorization rules as one matrix and is the reference for the FR-2 requirements.

Table 2.1: Permission matrix (C = create, R = read, U = update, D = delete; own = own records only; – = no access). Poster and Applicant columns refer to the gig in question.

Operation	Visitor	Student	Poster	Applicant	Admin
Register an account	C	–	–	–	–
Log in, log out	C	C, D	C, D	C, D	C, D
Browse open gigs	–	R	R	R	R
Read a gig's detail	–	R (open)	R (own, any state)	R (open)	R
Post a gig	–	C	C	C	C
Edit or delete a gig[p]	–	–	U, D	–	D
Apply to a gig	–	C	–	–	–
Read the applicant list	–	–	R	–	R[p]
Change an application's status	–	–	U	–	–
Read own applications and totals	–	own	own	own	own
Read or update own profile	–	own	own	own	own
Another student's email address	–	–	–	–	R[p]
Raise a dispute[p]	–	–	C	C	–
Resolve a dispute[p]	–	–	–	–	U
Read the ledger[p]	–	own	own	own	R

[p] Planned. The operation is specified in Chapter 3 but is not present in the current build.

2.3 Product Functions

The system stores four kinds of records today and will store several more. Figures 2.2 and 2.3 give the data model for the complete platform, split across two sheets for readability.

In Figure 2.2 a User posts many Gigs, places many Bids and holds Ratings. A Gig binds at most one accepted Contract, the point where an agreement becomes an obligation; that Contract owns its Deliverable records, its StatusHistory entries, at most one Dispute and up to two Ratings, one from each party. Filled diamonds mark composition: a status history row has no meaning apart from its contract. ReputationSummary is marked derived, recording that it is recomputed from settled contracts and is never an independent source of truth.

Every money attribute is a whole number of paise, never a floating point number, because binary floating point cannot hold a decimal currency value exactly and a rounding error in a ledger cannot be recovered.

Figure 2.3 shows the accounting design. A LedgerTransaction owns two or more LedgerEntry rows, each posted to an Account. Each user owns an available account and

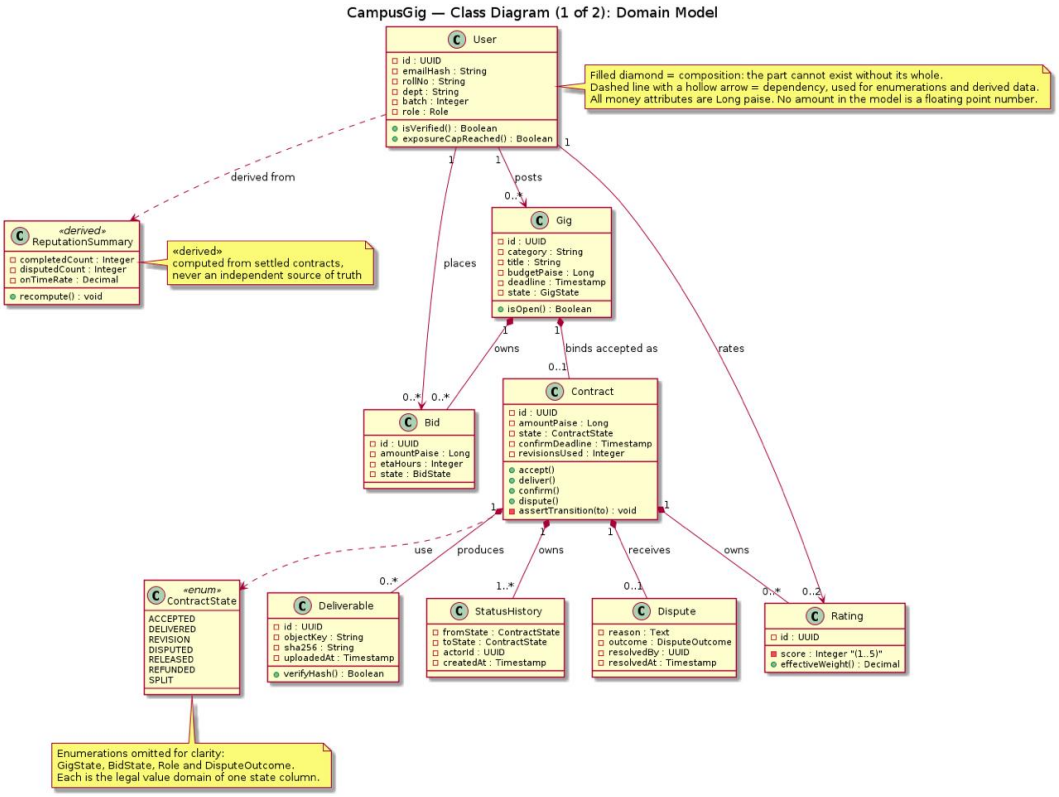


Figure 2.2: Domain model, part 1 of 2

a held account, and the platform owns one. Money entering escrow is not a flag on a row; it is a transaction taking an amount out of the poster's available account and putting the same amount into their held account, so the total never changes and a balance is always the sum of the entries posted to it. The two notes record the rules that make this safe: every transaction must sum to zero, and ledger entries can only be appended, so a correction is a new compensating transaction rather than an edit. IdempotencyKey stores the response against a client-supplied key, so a retried request returns the original answer instead of holding funds twice.

Figure 2.4 decomposes the system into five processes and six data stores. Process 1.0 validates credentials and maintains D1 Users and D2 Sessions, storing only a hash of the cookie value. Process 2.0 writes the short description and file paths to D1 and the files themselves to D3. Process 3.0 creates and lists gigs in D4, reading poster details from D1 so a listing can show who posted a gig without exposing their email. Process 4.0 writes applications to D5 and, when a poster awards a gig, writes the closure back to D4 in the same transaction. Process 5.0 and store D6 are dashed: specified here, not yet built. Table 2.2 gives the four stores that exist today with every column and constraint, and Table 2.3 the stores the planned features add.

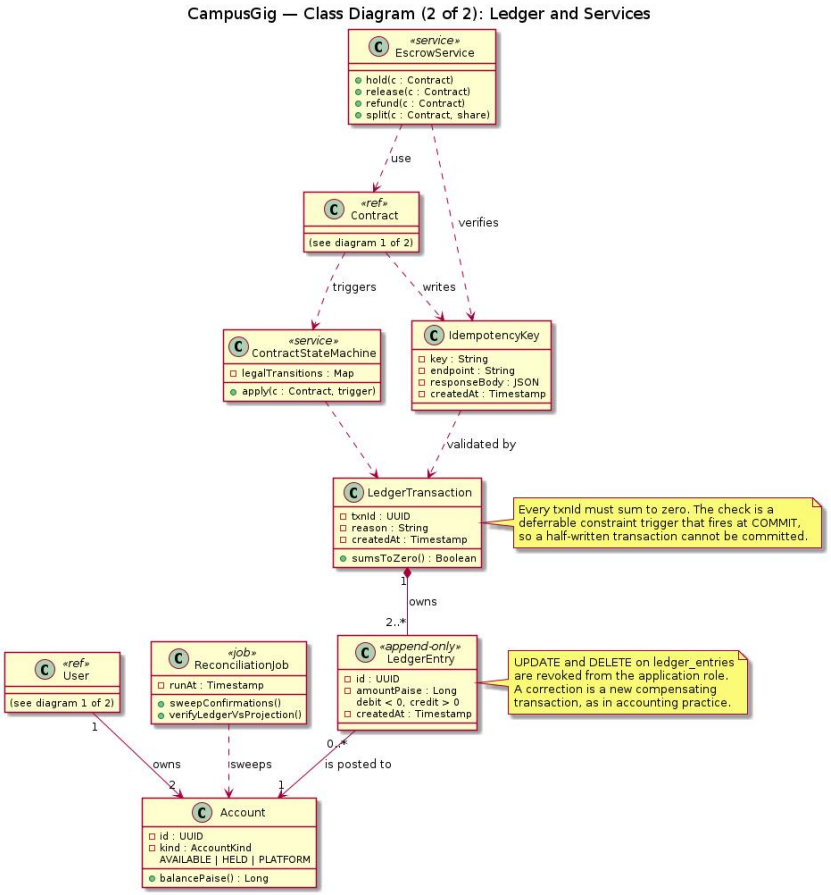


Figure 2.3: Ledger entities and services, part 2 of 2

Database schema

Three sequential Alembic migrations create the schema. All primary keys are UUIDs generated by the database, and all timestamps are timezone-aware and defaulted server-side.

Table 2.2: Current database schema and its constraints

Table	Columns and constraints
`users`	id; email (case-insensitive text, unique); email_hash (generated by the database from email, so the two cannot drift); password_hash; roll_no (unique); dept; batch (check 1900 to 2100); role (enum: student, admin); failed_login_attempts; locked_until; bio; avatar_path; resume_path; created_at; updated_at
`sessions`	id; user_id (foreign key to users, cascade on delete, indexed); token_hash (unique); expires_at (indexed); created_at

Table	Columns and constraints
`gigs`	id; poster_id (foreign key to users, restrict on delete, indexed); title (check length 5 to 200); description; category (enum of eight values); budget (numeric with two decimal places, check greater than zero); deadline; duration_days (check 1 to 365 when present); state (enum: open, closed); created_at; updated_at. Indexes on category and on (state, created_at descending)
`applications`	id; gig_id (foreign key to gigs, cascade on delete, indexed); applicant_id (foreign key to users, cascade on delete, indexed); message; status (enum: applied, accepted, rejected, completed); created_at; updated_at. Unique constraint on (gig_id, applicant_id); index on (applicant_id, status)

Table 2.3: Planned tables, derived from Figures 2.2 and 2.3

Table	Purpose and indicative columns
`contracts`	gig, worker, amount in paise, state, confirm_deadline, revisions_used
`status_history`	contract, from_state, to_state, actor, created_at
`deliverables`	contract, object_key, sha256, uploaded_at
`disputes`	contract, reason, outcome, resolved_by, resolved_at
`accounts`	owner, kind (available, held, platform)
`ledger_transactions`	reason, created_at
`ledger_entries`	transaction, account, amount in paise (negative for debit, positive for credit); append-only, with update and delete revoked from the application role
`idempotency_keys`	key, endpoint, response_body, created_at
`ratings`	contract, rater, score (1 to 5)

Data design rules

- A rule that can be a database constraint shall be a database constraint, not a check in application code.
- Monetary amounts shall be integer paise in the planned tables and a fixed-precision numeric in the current `gigs` table. Floating point is prohibited for money.
- Derived values shall be derivable. The email hash is a generated column, and the reputation summary must be recomputable from settled contracts.
- File storage shall hold only a relative path or an object key in the database, never an absolute path or a full URL, so the storage backend can change without a data migration.
- An operational change and its ledger effect shall be committed in the same database transaction.

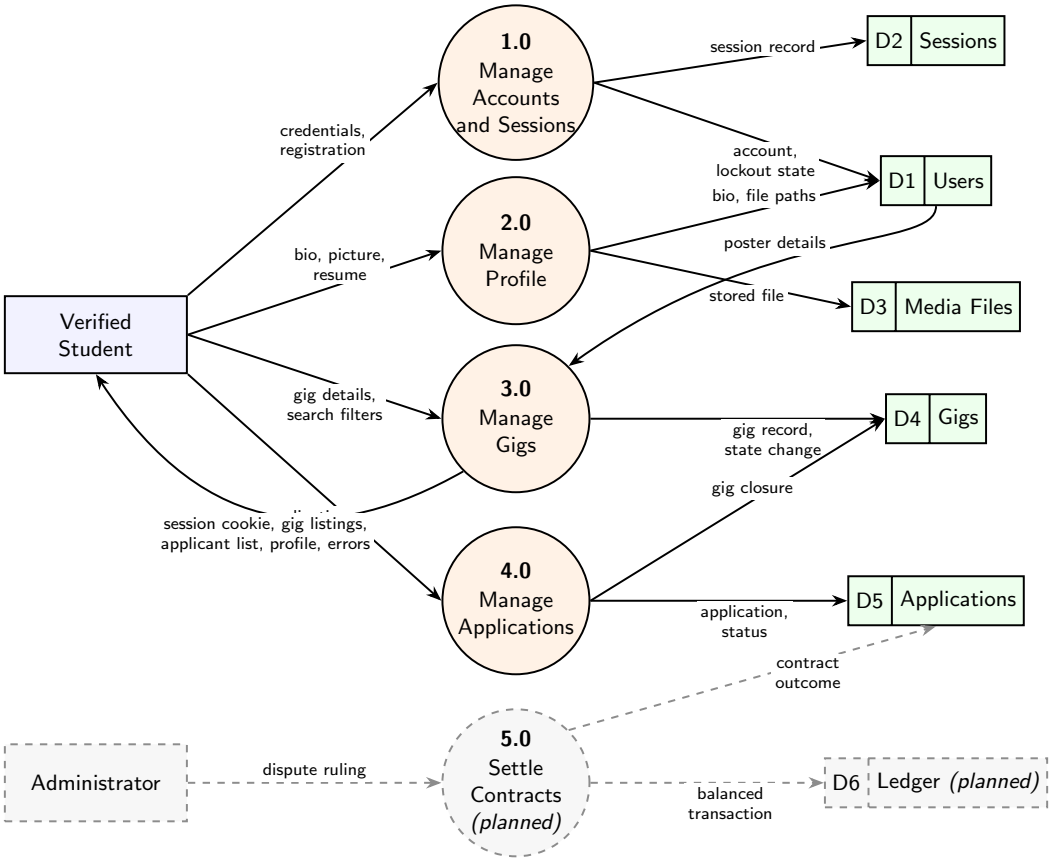


Figure 2.4: Data flow diagram (level 1)

2.4 Operating Environment

The browser application runs on any current desktop or mobile browser supporting the standard fetch interface and cookies, on Windows, macOS, Linux, Android or iOS, with no installation on the student's machine. The server component requires Python 3.12 or later on any operating system supporting it, and the database requires PostgreSQL 15 or later.

Uploaded pictures and resumes are written to a directory on the server's local disk. This is adequate for one server process and not for more, because two processes would not share a filesystem.

2.5 Design

Student activity splits into two flows, posting work and doing work, which meet when a poster awards a gig.

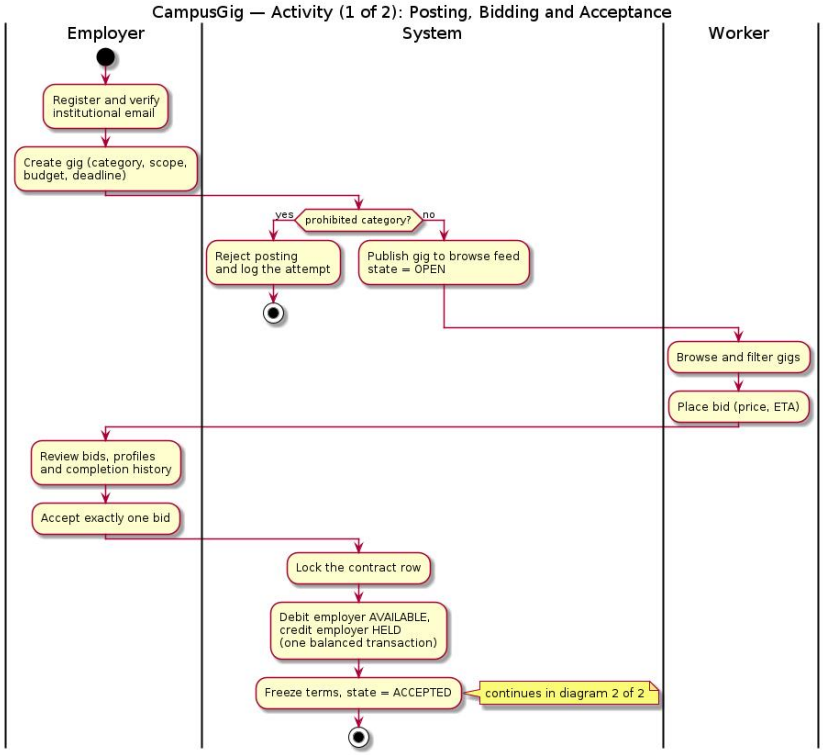


Figure 2.5: Activity diagram: posting, bidding and acceptance

Figure 2.5 partitions the first flow by responsibility. The employer registers, creates a gig, and the system either rejects the posting on category grounds or publishes it as open. A worker browses, filters and places a bid. The employer accepts exactly one, at which point the system locks the contract row, writes one balanced transaction moving the amount from available to held, and freezes the terms. The built system follows this as far as the acceptance decision: registration, gig creation, publication as open, filtered browsing, applying and the employer's decision all exist. Category screening does not, because the category is a validated list with no prohibited entries, and nor do the last three activities in the system lane, because there is no contract row to lock and no ledger to write to.

Figure 2.6 covers what follows acceptance and is entirely specification. The worker uploads a deliverable, the system records its storage key and hash and opens a confirmation window, and the employer reviews and may request a revision up to a limit. Settlement then takes one of three routes: the employer confirms, the sweeper releases automatically at expiry, or a dispute is raised and an administrator rules. All three con-

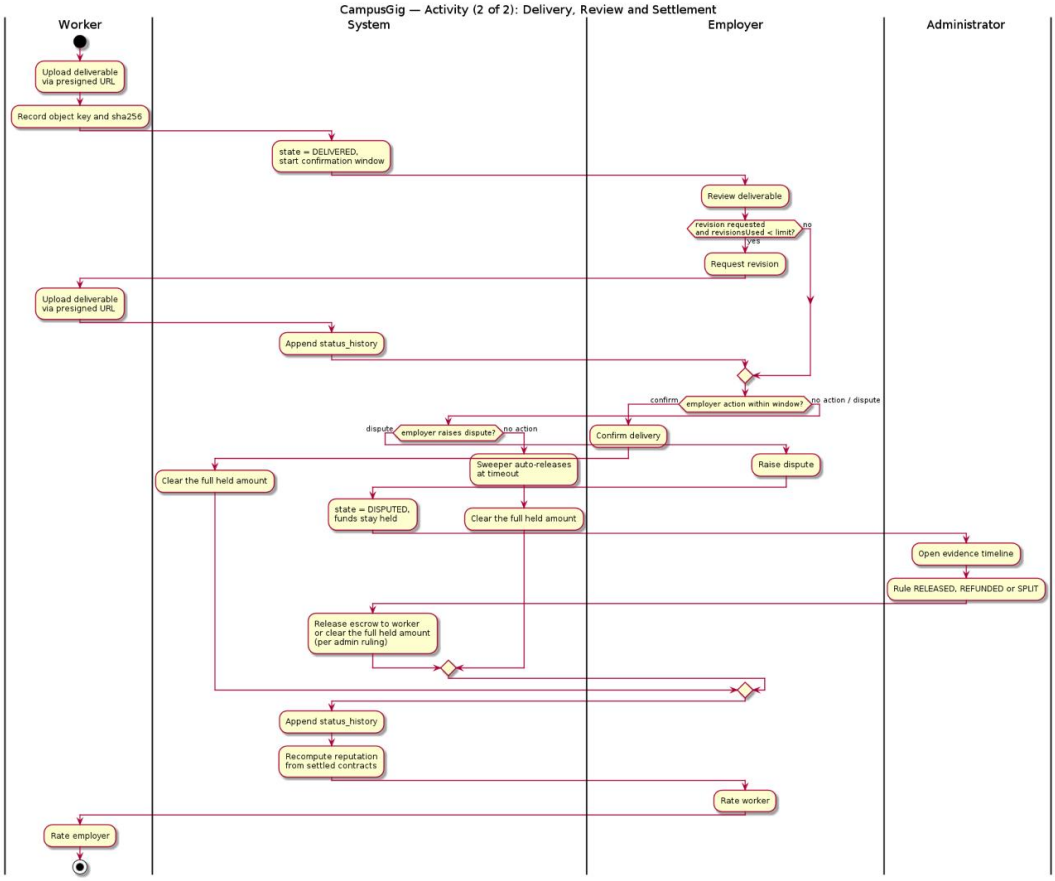


Figure 2.6: Activity diagram: delivery, review and settlement

verge on clearing the full held amount, appending to the status history and recomputing reputation. The only part existing today is the completed status a poster can set, which records that work was delivered but moves no money and stores no file.

Figure 2.7 specifies the order of operations for the acceptance step, and the order is what makes it correct. The API first looks up the idempotency key; if it has been seen, the stored response is replayed and no second hold occurs. Otherwise a database transaction begins, the contract row is locked, and only after the lock is held is the state read and checked, so two simultaneous acceptances cannot both pass. The escrow service then writes the balanced pair of entries, and the sum-to-zero rule is verified at commit.

Figure 2.8 specifies delivery and the one difficult race in the design. The upload half uses a short-lived signed address so the file goes straight to object storage and the API records only the key and hash. In the settlement half, the employer's confirmation and the scheduler's automatic release fire at the same instant. The confirming transaction takes the row lock first, so the sweeper blocks until it commits; when the sweeper proceeds it reads a state already released, the transition is rejected, and it returns a conflict without writing to the ledger. Exactly one settlement occurs, which is the property this

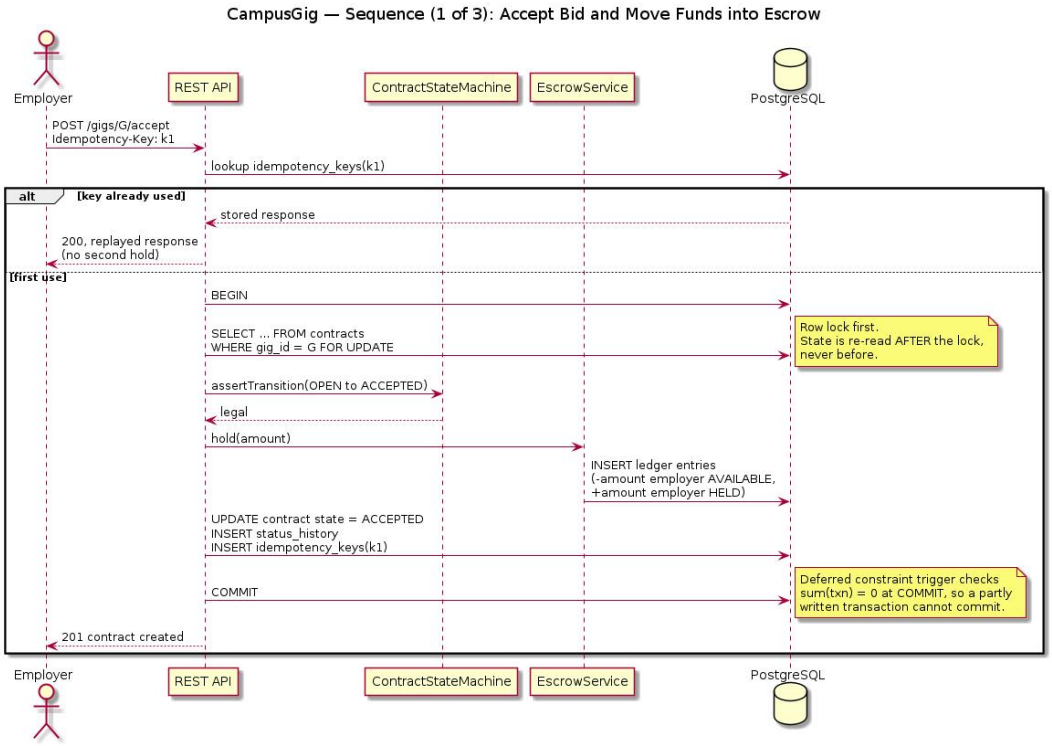


Figure 2.7: Sequence diagram: accepting a bid and moving funds into escrow

design must guarantee.

Figure 2.9 specifies the dispute path. Raising a dispute has no ledger effect: the contract is marked disputed and the funds stay in the held account, which is what makes the state safe to enter, since nothing is paid out while it is examined. The administrator's view is assembled from the status history, the deliverable records, the agreed terms and the deadlines, and carries the stored hash of each file so the administrator can confirm that what they downloaded is what was uploaded. Resolution takes the same row lock and writes one balanced transaction, which for a sixty-forty split is three entries summing to zero that clear the hold completely.

User interfaces

- **Guest pages:** signup and login. Reachable without a session; a signed-in visitor is redirected away from them.
- **Gigs board** (`/gigs`): the open gig listing with search, category, budget and deadline filters, sorting, pagination, and a button opening the post-a-gig dialog.
- **Gig detail** (`/gigs/:id`): the full description, with an apply form for other students and, for the poster, the applicant list with accept, decline and complete controls.

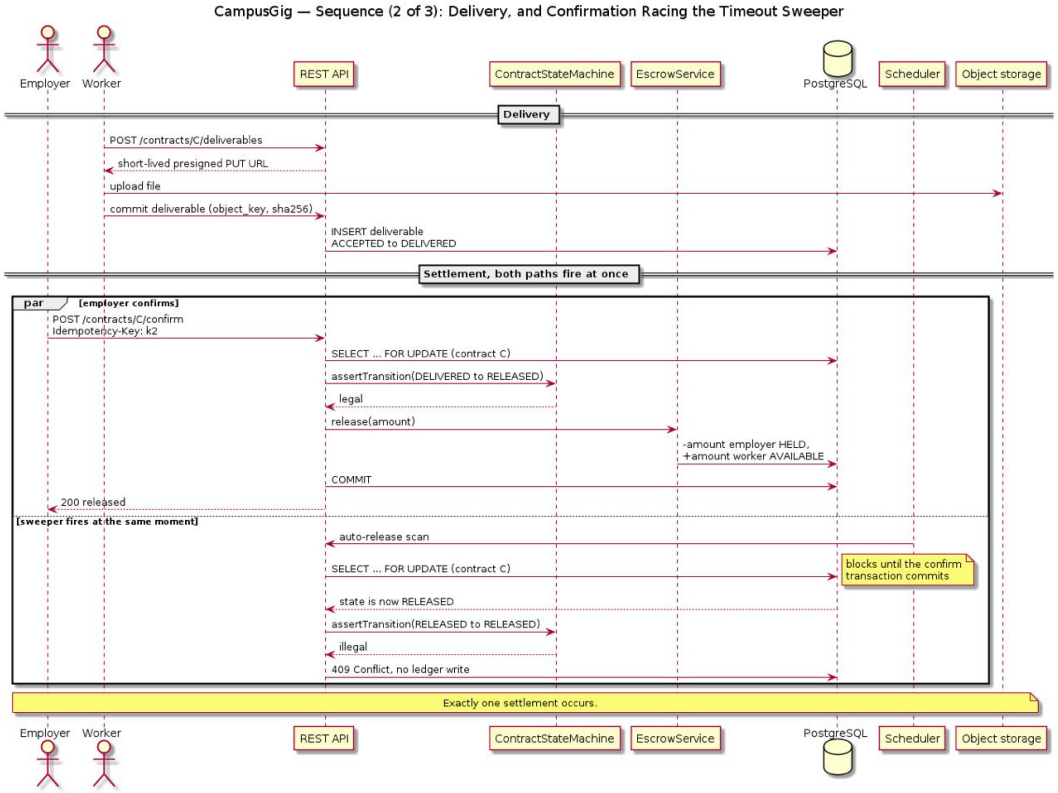


Figure 2.8: Sequence diagram: delivery, and confirmation racing the timeout sweeper

- **Applied page** (`/applied`): the student's applications with summary cards for completed value, in-progress value, total applications and replies pending, carrying the statement that no payment has taken place.
- **Profile page** (`/profile`): picture, resume, short description and read-only signup details.

Software interfaces

The endpoints below are the current contract; those marked session require a valid session cookie.

Table 2.4: REST API endpoints in the current build

Method	Path	Access	Purpose
GET	`/health`	none	Service liveness
GET	`/health/db`	none	Database round trip
POST	`/api/v1/auth/register`	none	Create an account; no session is issued

Method	Path	Access	Purpose
POST	/api/v1/auth/login	none	Create a session cookie
POST	/api/v1/auth/logout	session	Delete the session, clear the cookie
GET	/api/v1/auth/me	session	The authenticated student
GET	/api/v1/gigs	session	Paged, filtered list of open gigs
POST	/api/v1/gigs	session	Post a gig; poster is the session user
GET	/api/v1/gigs/mine	session	Gigs I posted, any state
GET	/api/v1/gigs/{id}	session	An open gig, or any of my own
POST	/api/v1/gigs/{id}/apply	session	Apply to a gig
GET	/api/v1/gigs/{id}/ applications	poster only	Who applied to my gig
GET	/api/v1/applications/me	session	My applications plus summary totals
PATCH	/api/v1/applications/ {id}	poster only	Accept, decline or complete
GET	/api/v1/profile	session	My profile
PATCH	/api/v1/profile	session	Update the short description
POST	/api/v1/profile/avatar	session	Upload a profile picture
POST	/api/v1/profile/resume	session	Upload a resume PDF
GET	/media/{path}	none	Uploaded files, served under generated names

The listing endpoint accepts `page`, `page_size`, `q`, `category`, `min_budget`, `max_budget`, `deadline_before` and `sort`, all validated at the boundary. Every successful list response is an envelope carrying `items`, `page`, `page_size`, `total` and `total_pages`. Every failure response is { `detail:` { `code, message` } }, with validation failures adding a `fields` array.

Communication interfaces

HTTP with JSON bodies between browser and API, and multipart form data for the two upload endpoints. The session travels as a cookie and the browser sends credentials on every request. The API reaches PostgreSQL over its own protocol using a connection string from environment configuration. HTTPS is assumed in production and the secure cookie flag is set accordingly. There is no message queue, no websocket and no outbound integration.

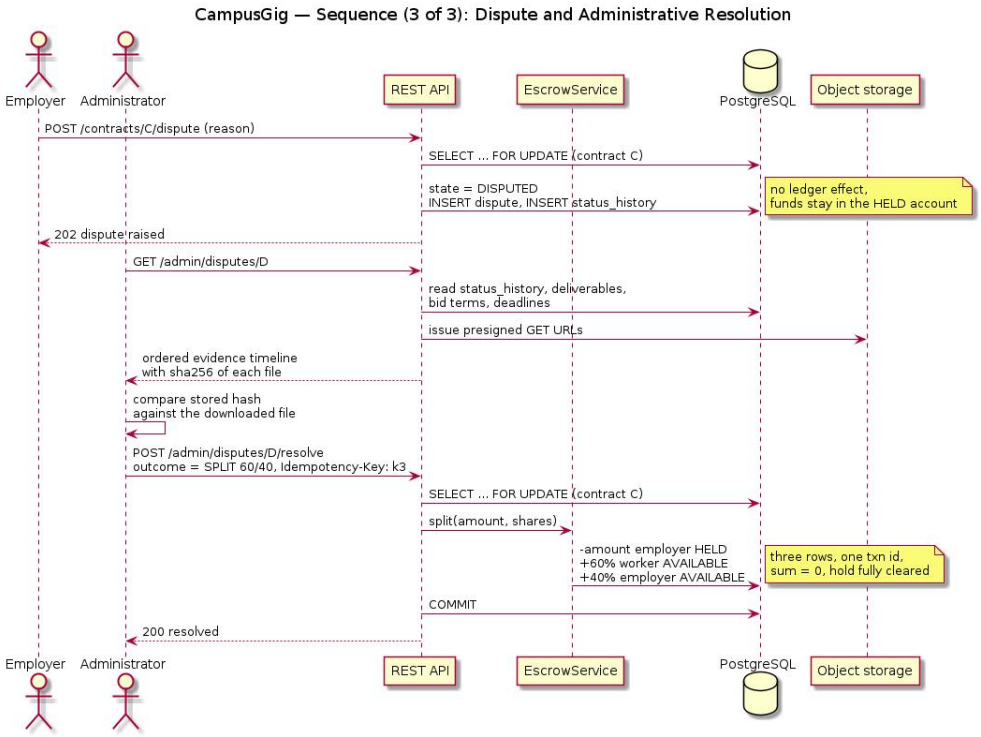


Figure 2.9: Sequence diagram: raising a dispute and resolving it from the evidence trail

3 System Features

3.1 Description and Priority

Priorities are **High** (essential), **Medium** (important) and **Low** (desirable). Each requirement also carries a status, checked against the current source code:

- **Implemented**: present and working in the current code base, and covered by at least one automated test.
- **Partial**: partly present; the requirement states what is missing.
- **Planned**: specified here, not claimed as implemented.
- **To verify**: a target value that must be measured before it can be claimed.

Table 3.1: System features, priorities and overall status

No.	Feature	Description	Priority	Status
F1	Authentication and Account Management	Institutional registration, login, logout, server-side sessions, login lockout.	High	Implemented
F2	Authorization and Record Scope	Server-side ownership checks and response field scoping.	High	Implemented
F3	Gig Management	Posting, browsing, filtering, sorting, paging, gig detail.	High	Partial
F4	Applications and Awarding	Applying, accepting, declining, completing, applied report.	High	Partial
F5	Profile and File Uploads	Picture, resume, short description, signup details.	High	Implemented
F6	Contracts and Lifecycle	Contract state machine, guards, status history.	High	Planned
F7	Escrow and Ledger	Holds, releases, double-entry ledger, idempotency.	High	Planned
F8	Deliverables, Disputes and Reputation	Evidence upload, dispute resolution, ratings.	Medium	Planned
F9	Error Reporting and System Health	Uniform error contract, status codes, health endpoints.	Medium	Implemented

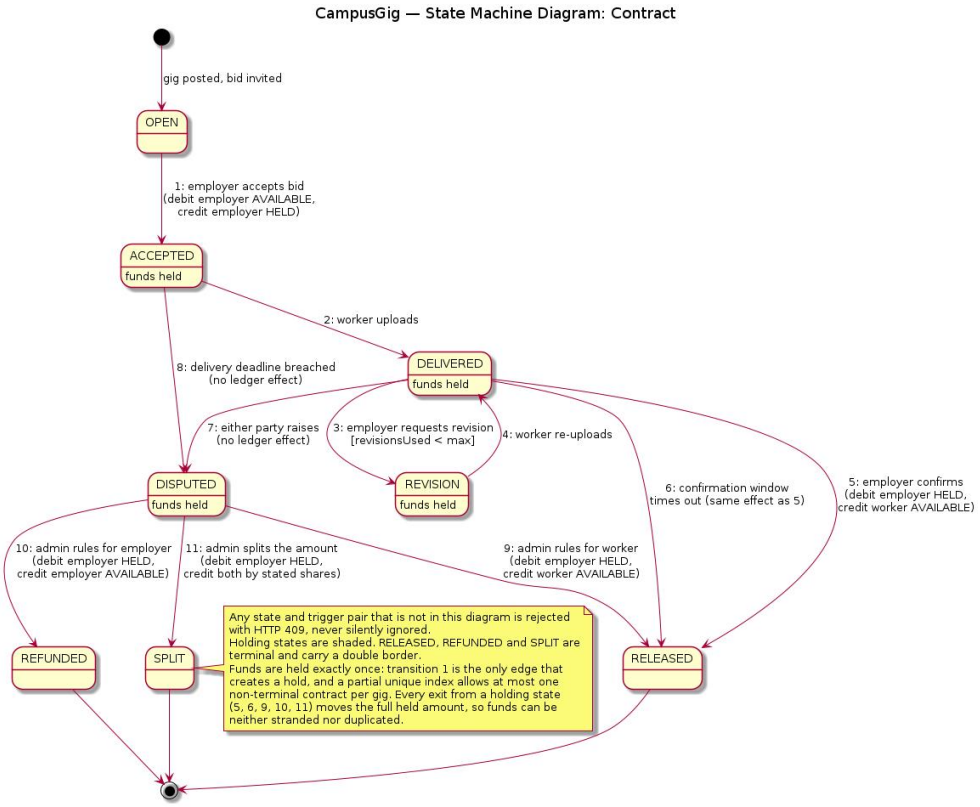


Figure 3.1: State machine diagram for a contract, with the ledger effect of each transition

The contract lifecycle governs F6 to F8, and its legal transitions are fixed by Figure 3.1.

A contract starts open and ends in one of three terminal states: released, refunded or split. Each labelled transition names its ledger effect, so the diagram doubles as the specification of what the escrow service must write. Transition 1 is the only edge creating a hold, and transitions 5, 6, 9, 10 and 11 each move the entire held amount out, which guarantees that funds can be neither stranded nor released twice. Any state and trigger combination not on this diagram must be rejected with a conflict response and never silently ignored. The application status flow that exists today is a smaller version of the same idea, specified in FR-4.4.

3.2 Functional Requirements

F1 – Authentication and Account Management

Stimulus and response: a student submits registration details or credentials; the server validates them, creates an account or a session, and returns the session token in a cookie.

Table 3.2: F1 requirements: authentication and account management

ID	Requirement	Priority	Status
FR-1.1	The system shall accept a registration request containing an email address, a password, a roll number, a department and a batch year.	High	Implemented
FR-1.2	The system shall accept a registration only if the domain of the submitted address is exactly `thapar.edu`, compared case-insensitively after trimming whitespace and splitting on the final `@`. Near-miss domains such as `fakethapar.edu`, `sub.thapar.edu` and `thapar.edu.fake.com` shall be rejected.	High	Implemented
FR-1.3	The system shall enforce the domain rule on the server. Any equivalent check in the browser is for immediate feedback only and shall not be relied upon.	High	Implemented
FR-1.4	The system shall require a password of at least 8 characters and at most 72 bytes, containing at least one letter and at least one digit.	High	Implemented
FR-1.5	The system shall require a roll number of 3 to 32 characters, a department of 2 to 100 characters, and a batch year between 1900 and 2100.	Medium	Implemented
FR-1.6	The system shall reject a registration whose email address or roll number is already in use, treating addresses differing only in letter case as the same address.	High	Implemented
FR-1.7	The system shall store only a hash of the password and shall never return a password or its hash in any response or write it to any log.	High	Implemented
FR-1.8	The system shall not create a session on registration. The student shall be directed to log in as a separate act.	Medium	Implemented
FR-1.9	The system shall authenticate a student from an email address and a password and, on success, create a server-side session and return its token in a cookie that page scripts cannot read.	High	Implemented
FR-1.10	The system shall store only a hash of the session token and shall never retain the token itself.	High	Implemented
FR-1.11	The system shall return an identical failure response, with identical timing, whether the submitted address matches no account or the password is wrong.	High	Implemented
FR-1.12	The system shall lock an account after 10 consecutive failed login attempts for 15 minutes, and shall reset the counter on a successful login.	High	Implemented

ID	Requirement	Priority	Status
FR-1.13	The system shall invalidate a session on logout by deleting it on the server, so that the token cannot be reused afterwards.	High	Implemented
FR-1.14	The system shall reject a request bearing an expired, deleted or forged session token.	High	Implemented
FR-1.15	The system shall expire a session automatically after a configured lifetime, defaulting to seven days.	Medium	Implemented
FR-1.16	The system shall provide an endpoint returning the authenticated student, so that the browser application can restore the signed-in state after a page reload.	Medium	Implemented
FR-1.17	The system shall confirm that a registrant controls the address they supplied, by a one-time verification step. This was built and removed from scope; reinstating it is required before the platform is used beyond the laboratory.	High	Planned

F2 – Authorization and Record Scope

Stimulus and response: every protected request is checked for a valid session, then for ownership of the record in question, before any data is read or changed.

Table 3.3: F2 requirements: authorization and record scope

ID	Requirement	Priority	Status
FR-2.1	Every protected operation shall enforce authentication on the server. A route guard in the browser shall be treated as a convenience and never as a security control.	High	Implemented
FR-2.2	A student's role shall not be settable through any request body. Registration shall always create a student account, and the registration schema shall contain no role field.	High	Implemented
FR-2.3	The poster of a new gig shall be the authenticated student and its state shall be open. Neither value shall be accepted from the request body.	High	Implemented
FR-2.4	Only the poster of a gig shall be able to read its applicant list or change the status of an application on it. An applicant shall not be able to promote their own application.	High	Implemented
FR-2.5	A student's email address shall never be disclosed to another student. Only the identifier, roll number, department and batch of a poster or applicant shall be visible.	High	Implemented

ID	Requirement	Priority	Status
FR-2.6	Responses shall be built from explicit response schemas. The password hash, the email hash and the lockout counters shall never be serialised.	High	Implemented
FR-2.7	An unauthorized request shall return 401 when not authenticated or 403 when not the owner, and shall not change stored data.	High	Implemented
FR-2.8	Administrator capabilities shall be assignable only by an existing administrator and shall be auditable.	Medium	Planned
FR-2.9	The permission rules in Table 2.1 shall be expressed in one place in the service layer so that they can be reviewed and tested as a whole.	Medium	Planned

F3 – Gig Management

Stimulus and response: a student posts a gig or requests a filtered page of open gigs; the server validates the input, applies the filters and returns a page with its totals.

Table 3.4: F3 requirements: gig management

ID	Requirement	Priority	Status
FR-3.1	The system shall allow an authenticated student to post a gig with a title of 5 to 200 characters, a description of 20 to 5000 characters, a category from the fixed list, a budget greater than zero and at most 1,000,000 to two decimal places, a deadline in the future, and an optional expected duration of 1 to 365 days.	High	Implemented
FR-3.2	The system shall return a paged list of open gigs, filterable by free text across title and description, by category, by minimum and maximum budget, and by latest deadline.	High	Implemented
FR-3.3	The listing shall be sortable by newest, by deadline, by ascending budget and by descending budget, with a page size between 1 and 50 defaulting to 12, and shall return the page number, page size, total count and total page count.	High	Implemented
FR-3.4	The system shall reject a listing request whose minimum budget exceeds its maximum budget, whose sort order is not one of the four permitted values, or whose page size exceeds the cap.	Medium	Implemented
FR-3.5	The system shall allow an authenticated student to retrieve a single open gig, and shall allow a poster to retrieve their own gig after it has been closed.	High	Implemented

ID	Requirement	Priority	Status
FR-3.6	The system shall allow a student to list the gigs they have posted, in any state.	Medium	Implemented
FR-3.7	The system shall allow a poster to edit or withdraw a gig that has not yet been awarded.	Medium	Planned
FR-3.8	The system shall screen a posted gig against a configured list of prohibited categories and refuse the posting if it matches.	Low	Planned

F4 – Applications and Awarding

Stimulus and response: a student applies to a gig, or its poster changes an application's status; the server checks ownership and the permitted transition, then records the change.

Table 3.5: F4 requirements: applications and awarding

ID	Requirement	Priority	Status
FR-4.1	The system shall allow an authenticated student to apply to an open gig with an optional message of at most 1000 characters.	High	Implemented
FR-4.2	The system shall reject an application to a gig that does not exist, to a gig the applicant posted, or to a gig that is not open, with a distinct error code in each case.	High	Implemented
FR-4.3	The system shall permit at most one application per student per gig, enforced as a database constraint rather than by checking before inserting, so that two concurrent submissions cannot both succeed.	High	Implemented
FR-4.4	The system shall permit only the following status changes: applied to accepted, applied to rejected, and accepted to completed. Rejected and completed shall be terminal, and every other requested change shall be refused as a conflict.	High	Implemented
FR-4.5	When an application is accepted, the system shall close the gig and set every other application on that gig still in the applied status to rejected, within the same database transaction.	High	Implemented
FR-4.6	The system shall return a student's applications and the summary counts describing them in a single response, so that the two cannot disagree on screen.	Medium	Implemented

ID	Requirement	Priority	Status
FR-4.7	The summary shall report the total budget of the student's completed applications and of their accepted applications, and the interface shall present these as the value of work and not as money that has moved.	High	Implemented
FR-4.8	The system shall allow an applicant to withdraw an application that has not yet been accepted.	Medium	Planned
FR-4.9	The system shall allow a priced bid and a counter-offer, so that an applicant may propose an amount other than the posted budget.	Medium	Planned

F5 – Profile and File Uploads

Stimulus and response: a student updates their description or uploads a picture or resume; the server validates the file by its content and stores it under a generated name.

Table 3.6: F5 requirements: profile and file uploads

ID	Requirement	Priority	Status
FR-5.1	The system shall return the authenticated student's own profile, comprising the signup details, the short description, and the addresses of the picture and resume if present.	High	Implemented
FR-5.2	The system shall allow a student to set or clear a short description of at most 1000 characters.	Medium	Implemented
FR-5.3	The system shall accept a profile picture in JPEG, PNG, GIF or WEBP form of at most 2 MB, and a resume in PDF form of at most 5 MB.	High	Implemented
FR-5.4	The system shall determine the type of an uploaded file from its leading bytes and shall not rely on the type declared by the browser.	High	Implemented
FR-5.5	The system shall enforce the size limit while the file is being received and shall discard a partial file that exceeds it.	High	Implemented
FR-5.6	The system shall store an uploaded file under a randomly generated name and shall never reuse the name supplied by the client.	High	Implemented
FR-5.7	When a file is replaced, the system shall record the new file before removing the old one, so that a profile never refers to a file that is absent.	Medium	Implemented
FR-5.8	The system shall store uploaded files in object storage rather than on local disk, and shall serve private files through an authenticated endpoint.	Medium	Planned

F6 – Contracts and Lifecycle

Stimulus and response: a gig is awarded; the system creates a contract and thereafter admits only the transitions drawn in Figure 3.1.

Table 3.7: F6 requirements: contracts and lifecycle

ID	Requirement	Priority	Status
FR-6.1	The system shall create a contract when a poster awards a gig, recording the agreed amount and freezing the terms.	High	Planned
FR-6.2	The system shall permit only the transitions shown in Figure 3.1 and shall reject every other state and trigger combination with a conflict response rather than ignoring it.	High	Planned
FR-6.3	The system shall record every contract transition in an append-only status history carrying the previous state, the new state, the acting user and the time.	High	Planned
FR-6.4	The system shall lock the contract row before reading its state on any transition, so that two concurrent requests cannot both pass the state check.	High	Planned
FR-6.5	The system shall open a confirmation window when a deliverable is submitted and shall settle automatically when that window expires without action.	High	Planned
FR-6.6	The system shall ensure that exactly one settlement occurs for a contract even when a confirmation and an automatic release are attempted simultaneously.	High	Planned
FR-6.7	The system shall permit at most one non-terminal contract per gig, enforced by a database constraint.	High	Planned

F7 – Escrow and Ledger

Stimulus and response: a contract transition moves money; the escrow service writes one balanced ledger transaction and the database verifies it at commit.

Table 3.8: F7 requirements: escrow and ledger

ID	Requirement	Priority	Status
FR-7.1	The system shall hold funds by writing a single transaction that debits the poster's available account and credits their held account, and shall create a hold by no other means.	High	Planned

ID	Requirement	Priority	Status
FR-7.2	Every ledger transaction shall sum to zero, verified by a deferred check at commit time, so that a partially written transaction cannot be stored.	High	Planned
FR-7.3	Ledger entries shall be appendable only. A correction shall be a new compensating transaction and never a modification or deletion of an existing entry.	High	Planned
FR-7.4	Every request that moves money shall accept an idempotency key, and a repeated key shall return the stored response without performing the operation again.	High	Planned
FR-7.5	All monetary amounts shall be stored as whole numbers of paise. No amount shall be represented as a floating point number anywhere in the system.	High	Planned
FR-7.6	An account balance shall be the sum of the entries posted to it and shall never be stored as an independently updatable number.	High	Planned
FR-7.7	Each student shall own one available account and one held account, and the platform shall own one account.	Medium	Planned

F8 – Deliverables, Disputes and Reputation

Stimulus and response: a worker submits evidence of completed work, or a party raises a dispute; the system records the evidence and, if required, presents it to an administrator for a ruling.

Table 3.9: F8 requirements: deliverables, disputes and reputation

ID	Requirement	Priority	Status
FR-8.1	The system shall allow a worker to upload a deliverable to object storage through a short-lived signed address, and shall record its storage key and content hash.	High	Planned
FR-8.2	The system shall allow a poster to request a revision up to a configured limit, returning the contract to the worker with no ledger effect.	Medium	Planned
FR-8.3	The system shall allow either party to raise a dispute, which shall move the contract to the disputed state and leave the held funds untouched.	High	Planned
FR-8.4	The system shall present an administrator with an ordered evidence timeline for a disputed contract, including the recorded content hash of every deliverable.	High	Planned

ID	Requirement	Priority	Status
FR-8.5	The system shall allow an administrator to resolve a dispute in favour of either party or by a stated split, writing one balanced transaction that clears the hold entirely.	High	Planned
FR-8.6	The system shall allow each party to rate the other once per settled contract, on a scale of 1 to 5.	Medium	Planned
FR-8.7	The system shall compute a reputation summary from settled contracts only, and shall be able to recompute it from those contracts at any time.	Medium	Planned
FR-8.8	The system shall allow an administrator to take down a prohibited gig and to produce a reconciliation report comparing ledger balances against expected values.	Medium	Planned

F9 – Error Reporting and System Health

Stimulus and response: any request fails, or an operator checks the service; the system answers in one consistent shape.

Table 3.10: F9 requirements: error reporting and system health

ID	Requirement	Priority	Status
FR-9.1	The system shall report every failure in a single consistent shape carrying a stable machine-readable code and a message suitable for display, including failures raised by the web framework itself.	High	Implemented
FR-9.2	The system shall use response codes consistently: 400 for a malformed request, 401 when not authenticated, 403 when not the owner, 404 when not found, 409 for a duplicate or a disallowed state change, 413 when a file is too large, 415 when a file type is not accepted, 422 when a field fails validation, and 429 when an account is locked.	High	Implemented
FR-9.3	The system shall provide an unauthenticated endpoint reporting that the service is running, and a second one confirming that the database can be reached.	Medium	Implemented
FR-9.4	The system shall publish an API document generated from the code itself, so that the documentation cannot drift from the endpoints.	Medium	Implemented

Verification and traceability

Test cases are written against requirement identifiers. The table below lists the automated tests that exist today, by file and test name, and the requirements each verifies; tests for *Planned* requirements have not been written and are not claimed as passing. The backend suite runs against a real PostgreSQL database that the harness creates and migrates with the project's own Alembic migrations, truncating between tests. The suite holds **103 backend test cases** and **20 frontend test cases**, counting parameterised cases individually.

Table 3.11: Requirement-to-test traceability for the implemented requirements

Test	Automated test	Verifies	Status
TC-01	`test_registration.py :: test_non_institutional_domains_are_rejected` (6 parameterised domains)	FR-1.2, FR-1.3, NFR-8	Passing
TC-02	`test_registration.py :: test_domain_check_is_case_insensitive_and_trims`	FR-1.2	Passing
TC-03	`test_registration.py :: test_weak_passwords_are_rejected` (5 cases)	FR-1.4	Passing
TC-04	`test_registration.py :: test_duplicate_email_differing_only_by_case_is_rejected`	FR-1.6, NFR-24	Passing
TC-05	`test_registration.py :: test_password_is_hashed_never_stored_plaintext`	FR-1.7, NFR-10	Passing
TC-06	`test_registration.py :: test_registration_does_not_log_the_user_in`	FR-1.8	Passing
TC-07	`test_registration.py :: test_role_cannot_be_injected_through_the_payload`	FR-2.2	Passing
TC-08	`test_auth_session.py :: test_successful_login_sets_an_httponly_cookie`	FR-1.9, NFR-12	Passing
TC-09	`test_auth_session.py :: test_nonexistent_account_is_indistinguishable_from_a_wrong_password`	FR-1.11, NFR-15	Passing
TC-10	`test_auth_session.py :: test_repeated_failures_lock_the_account`	FR-1.12	Passing
TC-11	`test_auth_session.py :: test_logout_invalidates_the_session`	FR-1.13	Passing
TC-12	`test_auth_session.py :: test_expired_session_is_rejected` and `test_me_with_a_forged_cookie_is_401`	FR-1.14, FR-1.15	Passing
TC-13	`test_auth_session.py :: test_protected_gig_endpoints_require_authentication`	FR-2.1, FR-2.7	Passing
TC-14	`test_applications.py :: test_poster_is_the_session_user_not_the_payload` and `test_state_cannot_be_set_through_the_payload`	FR-2.3	Passing

Test	Automated test	Verifies	Status
TC-15	`test_gigs.py :: test_listing_never_exposes_poster_email`	FR-2.5, FR-2.6	Passing
TC-16	`test_gigs.py :: test_pagination_metadata_and_slicing` and `test_page_size_is_capped`	FR-3.3, NFR-1, NFR-2	Passing
TC-17	`test_gigs.py :: test_search_matches_title_and_description`, `test_category_filter`, `test_budget_range_filter`, `test_deadline_filter`, `test_sorting`	FR-3.2, FR-3.3	Passing
TC-18	`test_gigs.py :: test_inverted_budget_range_is_rejected` and `test_invalid_query_parameters_are_rejected`	FR-3.4	Passing
TC-19	`test_gigs.py :: test_closed_gig_is_not_reachable_by_id` and `test_applications.py :: test_poster_can_still_see_their_own_closed_gig`	FR-3.5	Passing
TC-20	`test_applications.py :: test_invalid_gig_payloads_are_rejected`	FR-3.1	Passing
TC-21	`test_applications.py :: test_my_gigs_lists_only_mine`	FR-3.6	Passing
TC-22	`test_applications.py :: test_cannot_apply_to_own_gig` and `test_cannot_apply_to_a_closed_gig`	FR-4.2	Passing
TC-23	`test_applications.py :: test_cannot_apply_twice`	FR-4.3, NFR-24	Passing
TC-24	`test_applications.py :: test_invalid_status_transitions_are_refused`	FR-4.4	Passing
TC-25	`test_applications.py :: test_accepting_closes_the_gig_and_rejects_the_rest`	FR-4.5, BR-3	Passing
TC-26	`test_applications.py :: test_only_the_poster_can_change_status` and `test_only_the_poster_can_list_applicants`	FR-2.4, BR-4	Passing
TC-27	`test_applications.py :: test_my_applications_reports_totals` and `test_earnings_count_completed_work_only`	FR-4.6, FR-4.7	Passing
TC-28	`test_profile.py :: test_content_type_header_is_not_trusted` and `test_pdf_rejected_as_avatar_and_image_rejected_as_resume`	FR-5.4	Passing
TC-29	`test_profile.py :: test_oversized_upload_is_rejected`	FR-5.3, FR-5.5	Passing
TC-30	`test_profile.py :: test_replacing_an_avatar_removes_the_old_file`	FR-5.7	Passing

Test	Automated test	Verifies	Status
TC-31	`test_profile.py :: test_profile_never_exposes_credentials`	FR-2.6, FR-5.1	Passing
TC-32	`test_health_and_model.py :: test_app_starts_and_health_responds` and `test_database_health`	FR-9.3	Passing
TC-33	`test_health_and_model.py :: test_openapi_document_builds`	FR-9.4	Passing
TC-34	`test_health_and_model.py :: test_email_hash_is_generated_by_the_database` and `test_batch_check_constraint`	NFR-23, NFR-24	Passing
TC-35	`test_health_and_model.py :: test_production_refuses_default_auth_secret`	NFR-17	Passing
TC-36	`validation.test.ts :: isInstitutionalEmail` (12 cases) and `passwordError` (4 cases)	FR-1.2, FR-1.4 (browser feedback)	Passing
TC-37	`auth.test.tsx :: protected route` (2 cases)	NFR-27	Passing

Requirements under F6, F7 and F8, together with FR-1.17, FR-2.8, FR-2.9, FR-3.7, FR-3.8, FR-4.8, FR-4.9 and FR-5.8, have no tests because they have no implementation. NFR-5 and NFR-6 require a load test that has not been performed.

4 Other Nonfunctional Requirements

4.1 Performance Requirements

Table 4.1: Performance requirements

ID	Requirement	Priority	Status
NFR-1	The system shall cap the number of records returned by any list request at 50, so that no single request can ask the database for an unbounded result.	High	Implemented
NFR-2	Every list endpoint shall be paged and shall return the total count alongside the page, so that the browser can show how many results exist without fetching them.	High	Implemented
NFR-3	The gig listing shall be served by indexed queries. Indexes shall exist on the poster, on the category, and on the combination of state and creation time used by the default ordering.	Medium	Implemented
NFR-4	Free text search over gig titles and descriptions is a pattern scan, which is acceptable at the current data volume. It shall be replaced with a full-text index once the gig table approaches ten thousand rows.	Medium	Planned
NFR-5	Under laboratory load of up to 50 concurrent students, 95 percent of API requests shall complete within one second. This target shall be measured, not assumed.	Medium	To verify
NFR-6	The gigs board shall render its first page of results within three seconds in a documented test environment.	Medium	To verify
NFR-7	Login response time is deliberately bounded below by the bcrypt work factor, which is a security requirement (NFR-13) and not a performance defect. The work factor shall be reviewed if login latency becomes a usability problem.	Low	Implemented

No performance benchmarking has been carried out. No load generator has been run and no response time or throughput figure measured, so NFR-5 and NFR-6 are targets marked *To verify*, not results.

4.2 Security Requirements

Table 4.2: Security requirements

ID	Requirement	Priority	Status
NFR-8	Access to the system shall be restricted to holders of institute email addresses, enforced by the server-side domain check specified in FR-1.2.	High	Implemented
NFR-9	With email verification out of scope, the domain check is the entire access control and the system does not establish that a registrant controls the address they supplied. This is an accepted limitation of version 1.0 and is resolved by FR-1.17.	High	Partial
NFR-10	Passwords shall be stored using bcrypt and shall never appear in a response, a log line or an exception message.	High	Implemented
NFR-11	Session tokens shall be generated on the server with a cryptographically secure random source, shall be at least 256 bits, and shall be stored only as a hash.	High	Implemented
NFR-12	The session cookie shall be inaccessible to page scripts, shall be restricted to same-site requests, and shall be marked secure in production. No authentication material shall be placed in browser storage.	High	Implemented
NFR-13	Password hashing shall use a work factor of at least 12, so that an offline attack on a stolen hash remains expensive.	High	Implemented
NFR-14	The system shall accept cross-origin requests only from an explicitly configured list of origins and shall never use a wildcard origin while accepting credentials.	High	Implemented
NFR-15	The system shall not reveal whether an email address is registered, through either the content or the timing of a failed login response.	High	Implemented
NFR-16	The system shall construct every database query through parameter binding and shall never assemble a query by string concatenation.	High	Implemented
NFR-17	All secret values shall be supplied through the environment. The system shall refuse to start in production while the development default signing secret is in use, and no real secret shall be committed to the repository.	High	Implemented

ID	Requirement	Priority	Status
NFR-18	Uploaded files are served from unguessable addresses with no access check, which means the address itself is the permission. This is acceptable for a profile picture and shall not be used for deliverables, which must be placed behind an authenticated endpoint before FR-8.1 is implemented.	High	Partial
NFR-19	Rate limiting by network address shall be applied to the authentication endpoints, using a shared store so that it holds across processes.	Medium	Planned
NFR-20	Administrative and money-moving actions shall be recorded in an audit log that ordinary users cannot modify.	Medium	Planned
NFR-21	Uploaded files shall be scanned for malicious content before being served.	Low	Planned

4.3 Software Quality Attributes

Table 4.3: Software quality attributes

ID	Requirement	Priority	Status
NFR-22	**Testability:** business rules shall live in a layer that does not import the web framework, so that a rule can be tested without constructing a request.	High	Implemented
NFR-23	**Schema fidelity:** automated tests shall run against a real PostgreSQL database with the project's own migrations applied, so that a migration which does not match the code is caught by the suite.	High	Implemented
NFR-24	**Correctness by construction:** where a rule can be expressed as a database constraint it shall be. The one-application-per-gig rule, the case-insensitive uniqueness of email addresses, the derived email hash, and the bounds on title length, budget and batch year are all enforced by the database.	High	Implemented
NFR-25	**Maintainability:** dependencies between layers shall point in one direction. Routers shall contain no queries, services no framework types, and all queries shall be confined to the repository layer.	High	Implemented
NFR-26	**Modifiability:** all filesystem access shall be confined to a single module, so that moving to object storage is a change to one file.	Medium	Implemented

ID	Requirement	Priority	Status
NFR-27	**Usability:** the browser application shall restore the signed-in state after a page reload without showing the login page in the interim, and shall clear its local view of the user as soon as the server reports the session is no longer valid.	Medium	Implemented
NFR-28	**Honesty of presentation:** no feature from an unimplemented milestone shall appear in the interface as though it worked, and any monetary figure shown shall state that no payment has taken place.	High	Implemented
NFR-29	**Portability:** the browser application shall run on current desktop and mobile browsers without installation, and the server shall run on any platform supporting its language runtime.	Medium	Implemented
NFR-30	**Reliability under concurrency:** concurrent updates to the same record shall not silently overwrite one another. Today this is achieved for applications by a database constraint; contract transitions require the row locking of FR-6.4.	High	Partial
NFR-31	**Traceability:** every requirement shall be traceable to at least one test case, as recorded in the traceability table closing Section 3.2.	Medium	Partial
NFR-32	**Extensibility:** adding a gig category or a new sort order shall require changing a single enumeration or mapping, with no change to the query-building logic.	Medium	Implemented

4.4 Business Rules

- **BR-1** Only students of Thapar Institute may use CampusGig. Membership is established by the email domain and by nothing else.
- **BR-2** A student may act as a poster and as a worker at the same time, on different gigs. A student may never apply to a gig they posted.
- **BR-3** A gig may be awarded to at most one student. Awarding it closes the gig to further applications.
- **BR-4** Only the student who posted a gig may award it, decline an applicant, or mark the work complete.
- **BR-5** A gig is offered at a fixed budget. In the current version there is no negotiation; a student applies at the posted price and the poster answers.
- **BR-6** The system does not handle money. Any figure shown as earned is the budget of work recorded as complete and is not a payment.

- **BR-7** An administrator account may never be created through registration.
- **BR-8** Once the ledger exists, every movement of value shall be a balanced transaction, and no balance shall be stored as an independently updatable number.
- **BR-9** Raising a dispute never moves money. Funds stay held until a settlement or a ruling.
- **BR-10** An administrator may rule on a dispute but may not be a party to it.

5 Other Requirements

5.1 Constraints

- **PostgreSQL is not interchangeable.** The schema uses a case-insensitive text type for the email column, server-side generation of unique identifiers, and a generated column deriving the email hash inside the database. Another engine would require replacing all three.
- **One server process.** Uploaded files live on local disk, so a second instance would not see the first's files. Horizontal scaling requires object storage first.
- **Identity is borrowed.** The institute controls the email domain, so a student who loses their address loses access, and a graduate keeps access while the institute keeps their address.
- **No email verification.** It was built and then removed from scope, making the domain check the whole of the access control. The consequence is recorded in NFR-9.
- **Sessions, not tokens.** A stateless token cannot be revoked without a denylist table, which is more machinery than storing the session. This is fixed by the requirement that logout genuinely invalidates a session.
- **Academic project constraints.** The system runs on development machines and a local or containerised database, with no production deployment, monitoring stack or backup regime.

5.2 Assumptions and Dependencies

- An institute email address identifies exactly one student and is not shared.
- Transport is encrypted in production. The session cookie's protection depends on it.
- The number of gigs stays modest enough for a pattern-matching search, as recorded in NFR-4.
- Students will judge applicants from a profile and a message, which is the assumption behind having no reputation system in the current build.

- The planned features depend on components not yet selected or deployed: an object store for deliverables, a scheduler for time-based events, and a shared store for rate limiting.

5.3 Deferred Requirements

These are recognised requirements of the full platform that are not implemented in this version, listed so that each is a known gap rather than an oversight.

- Email ownership verification (FR-1.17), which was built and then removed from scope.
- Gig editing and withdrawal (FR-3.7), and application withdrawal (FR-4.8).
- Priced bidding and counter-offers (FR-4.9).
- All contract, escrow, ledger, deliverable, dispute, rating and administration requirements (F6, F7, F8).
- Object storage for uploads (FR-5.8), which is also a precondition for running more than one server process.
- Operational security controls: rate limiting, audit logging and upload scanning (NFR-19 to NFR-21).
- Full-text search for gigs (NFR-4).

Beyond the eight milestones, worthwhile enhancements are in-app notifications when an application changes status, saved searches or alerts on the gig board, exportable reports of completed work, and support for more than one institution through a configurable domain list.

Delivery order

1. **Milestone 1, complete.** Backend foundation, institutional signup, login, logout, server-side sessions, protected routes, and gig browsing with filters and pagination.
2. **Milestone 2, complete apart from FR-3.7 and FR-5.8.** Gig posting, poster ownership rules, categories, and profile files.
3. **Milestone 3, complete apart from FR-4.8 and FR-4.9.** Applications, poster decisions, and the applied report.
4. **Milestone 4.** Contracts: the state machine of Figure 3.1, status history, and transition guards (F6).
5. **Milestone 5.** Escrow and ledger: accounts, balanced transactions, the sum-to-zero constraint, idempotency keys and row locking (F7).

6. **Milestone 6.** Deliverables and disputes: object storage, signed upload addresses, content hashes, the evidence timeline, and the scheduler for confirmation timeouts (FR-8.1 to FR-8.5).

7. **Milestone 7.** Ratings and reputation (FR-8.6, FR-8.7).

8. **Milestone 8.** Administration: takedowns, ledger reconciliation and the audit log (FR-8.8, FR-2.8, NFR-20).

Acceptance criteria

Version 1.0 is accepted when the following hold; items 1 to 6 are already satisfied.

1. A new student with an institute address can sign up, land on the login page, log in, and log out in a way that invalidates the session on the server.

2. A signed-in student can post a gig and find it again through search, category, budget and deadline filters, with sorting and pagination.

3. A student can apply to another student's gig and cannot apply twice or apply to their own.

4. A poster can accept exactly one applicant, which closes the gig and declines the others in one transaction.

5. No secret, password or password hash appears in any API response or log line, and no student can read another student's email address.

6. `alembic upgrade head` on an empty database produces the full schema, and the complete automated suite passes against it.

7. A load test is run and NFR-5 and NFR-6 are either met or restated with measured values.

8. Email verification (FR-1.17) is reinstated before the system is used outside the laboratory.

5.4 Maintenance

CampusGig is intended to run across academic sessions and will need maintenance, starting with the category list as students use it. Each milestone in the delivery order of Section 5.3 changes the status of the requirements it implements, and this document is to be revised at each one. The targets marked *To verify* are to be replaced with measured values once a load test has been carried out, and the acceptance criteria updated accordingly.